

Q1. What are SQL Constraints?

Answer:

Constraints are rules applied to columns or tables to ensure data accuracy, integrity and reliability.

Common SQL Constraints:

Constraint	Description	Example
PRIMARY KEY	Uniquely identifies each row in a table. Cannot be NULL.	PRIMARY KEY (EmpID)
FOREIGN KEY	Maintains referential integrity between tables.	FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
UNIQUE	Ensures all values in a column are unique.	UNIQUE (Email)
NOT NULL	Ensures a column cannot have NULL values.	Name VARCHAR(100) NOT NULL
CHECK	Ensures all values satisfy a specific condition.	CHECK (Salary > 0)
DEFAULT	Sets a default value if none is provided.	Status VARCHAR(20) DEFAULT 'Active'
AUTO_INCREMENT / IDENTITY	Automatically generates a unique value for a column.	EmpID INT AUTO_INCREMENT

Example: Creating Table with Constraints

```
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY AUTO_INCREMENT,  
    EmpName VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Salary DECIMAL(10,2) CHECK (Salary > 0),  
    DeptID INT,  
    JoinDate DATE DEFAULT (CURRENT_DATE),  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

Key Points

- Constraints help maintain data integrity.
- PRIMARY KEY & FOREIGN KEY are the most important.
- CHECK & DEFAULT improve data quality.
- Use NOT NULL to avoid missing data.

Q2. What is ORDER BY, LIMIT and Pagination?

Answer:

ORDER BY is used to sort the result set.

LIMIT is used to restrict the number of rows returned.

Pagination is the process of fetching data in pages.

1. ORDER BY → Sorts the result set.

```
-- Ascending Order (Default)
SELECT * FROM Employee
ORDER BY Salary ASC;           (Low to High)

-- Descending Order
SELECT * FROM Employee
ORDER BY Salary DESC;         (High to Low)
```

2. LIMIT → Restricts number of rows.

```
-- Get first 10 rows
SELECT * FROM Employee
LIMIT 10;                     (Returns first 10 rows)

-- Get first 20 rows
SELECT * FROM Employee
LIMIT 20;                     (Returns first 20 rows)
```

Note: Different databases use different syntax.

- MySQL / PostgreSQL → **LIMIT**
- SQL Server → **TOP**
- Oracle 12c+ → **FETCH FIRST**

3. OFFSET → Skips rows.

```
-- Skip first 20 rows
SELECT * FROM Employee      (Skips first 20 rows
LIMIT 10 OFFSET 20;         and returns next 10 rows)
```

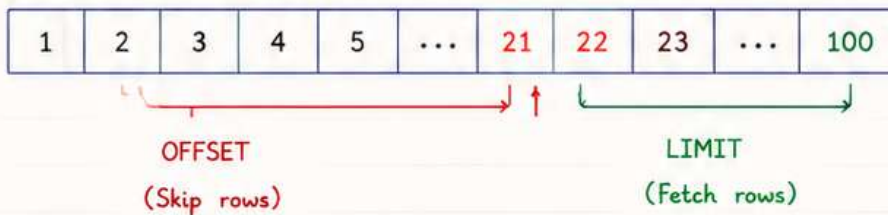
4. TOP (SQL Server)

```
SELECT TOP 10 *
FROM Employee
ORDER BY Salary DESC; (Returns top 10 rows)
```

5. FETCH FIRST (Oracle / PostgreSQL)

```
SELECT * FROM Employee
ORDER BY Salary DESC      (Returns first
FETCH FIRST 10 ROWS ONLY; 10 rows only)
```

Pagination (How it works?)



Examples:

- Page Size = 10
- Page Number = 1
(First Page)

```
SELECT * FROM Employee
LIMIT 10 OFFSET 0;
```

- Page Size = 10
- Page Number = 2
(Second Page)

```
SELECT * FROM Employee
LIMIT 10 OFFSET 10;
```

- Page Size = 10
- Page Number = 3
(Third Page)

```
SELECT * FROM Employee
LIMIT 10 OFFSET 20;
```

Key Points

- ORDER BY sorts data.
- LIMIT restricts rows.
- OFFSET skips rows.
- Used together for pagination.

Pro Tip: Always use ORDER BY with LIMIT for consistent and predictable results.

Q3. Aggregate Functions Deep Dive

Answer:

Aggregate functions perform calculations on a set of values and return a single value.

Function	Description	Example
COUNT()	Returns number of rows	SELECT COUNT(*) FROM Employee;
SUM()	Returns total sum	SELECT SUM(Salary) FROM Employee;
AVG()	Returns average value	SELECT AVG(Salary) FROM Employee;
MAX()	Returns maximum value	SELECT MAX(Salary) FROM Employee;
MIN()	Returns minimum value	SELECT MIN(Salary) FROM Employee;
COUNT(DISTINCT)	Counts unique values	SELECT COUNT(DISTINCT DeptID) FROM Employee;

Example Table: Employee

EmpID	EmpName	DeptID	Salary
1	Aman	10	60000
2	Neha	20	75000
3	Rahul	10	60000
4	Priya	30	90000
5	Vikas	20	75000

Sample Queries & Results

- SELECT COUNT(*) FROM Employee; -- 5
- SELECT SUM(Salary) FROM Employee; -- 360000
- SELECT AVG(Salary) FROM Employee; -- 72000
- SELECT MAX(Salary) FROM Employee; -- 90000
- SELECT MIN(Salary) FROM Employee; -- 60000
- SELECT COUNT(DISTINCT DeptID)
FROM Employee; -- 3

Key Points

- Aggregate functions are used with GROUP BY clause.
- COUNT(*) counts rows including NULL values.
- Other functions (SUM, AVG, MAX, MIN) ignore NULL values.
- COUNT(DISTINCT) counts only unique non-NULL values.

Q5. What are SQL Views?

Answer:

A view is a virtual table based on the result of a SQL query.

It does not store data physically.

1. Create View

```
CREATE VIEW view_name AS
  SELECT column1, column2, ...
  FROM table_name
  WHERE condition;
```

2. Use View

```
SELECT * FROM view_name;
SELECT column1, column2
  FROM view_name
  WHERE condition;
```

3. Example

Table: Employee

EmpID	EmpName	DeptID	Salary
1	Aman	10	60000
2	Neha	20	75000
3	Rahul	10	50000
4	Priya	30	90000

Step 1: Create View

```
CREATE VIEW HighSalaryEmp AS
  SELECT EmpID, EmpName, Salary
  FROM Employee
  WHERE Salary >= 60000;
```

Step 2: Query the View

```
SELECT * FROM HighSalaryEmp;
```

Result:

EmpID	EmpName	Salary
1	Aman	60000
2	Neha	75000
4	Priya	90000

4. Types of Views

- Simple View - Based on a single table.
- Complex View - Based on multiple tables or joins.
- Updatable View - Can be used to insert/update/delete data.
- Read-only View - Cannot be used to modify data.

5. Advantages of Views

- Simplifies complex queries.
- Enhances security (restricts access to data).
- Provides data abstraction.
- Makes queries reusable and easier to maintain.

6. Limitations of Views

- May impact performance.
- Not all views are updatable.
- Changes in base table affect the view.
- Cannot create index on a view.

Key Points

- A view is a virtual table based on a query result.
- It does not store data physically.
- Improves security, abstraction and reusability.
- Use views to simplify access to data and complex logic.

Q6. What are Indexes in SQL?

Answer:

Indexes are used to improve the speed of data retrieval operations on a table. They work like an index in a book.

1. Why use Indexes?

- Improves SELECT query performance.
- Reduces number of disk I/O operations.
- Speeds up JOINS, WHERE, ORDER BY, GROUP BY.
- Useful for large tables.

2. Types of Indexes

Index Type	Description
PRIMARY KEY	Automatically creates a unique clustered index.
UNIQUE INDEX	Ensures all values are unique.
CLUSTERED INDEX	Sorts and stores the actual data rows. Only one per table.
NON-CLUSTERED INDEX	Stores a separate structure with pointers to the data rows.
COMPOSITE INDEX	Index on multiple columns.

3. Syntax

-- Create Index

```
CREATE INDEX index_name  
ON table_name (column_name);
```

-- Create Composite Index

```
CREATE INDEX index_name  
ON table_name (column1, column2);
```

-- Drop Index

```
DROP INDEX index_name ON table_name;
```

4. Example

Step 1: Create Table

```
CREATE TABLE Employee (  
  EmpID    INT PRIMARY KEY,  
  EmpName  VARCHAR(100),  
  DeptID   INT,  
  Salary   DECIMAL(10,2)  
);
```

Step 2: Insert Data

```
INSERT INTO Employee VALUES  
(1, 'Aman', 10, 60000),  
(2, 'Neha', 20, 75000),  
(3, 'Rahul', 10, 50000),  
(4, 'Priya', 30, 90000);
```

Step 3: Create Index

```
CREATE INDEX idx_emp_name  
ON Employee (EmpName);
```

Step 4: Use Index in Query

```
SELECT * FROM Employee  
WHERE EmpName = 'Rahul';
```

5. How Index Works? (Example)

Without Index (Full Table Scan)	With Index																											
Check Each Row	Index on EmpName																											
<table border="1"> <tr><td>1</td><td>Aman</td></tr> <tr><td>2</td><td>Neha</td></tr> <tr><td>3</td><td>Rahul</td></tr> <tr><td>4</td><td>Priya</td></tr> <tr><td>...</td><td>...</td></tr> <tr><td>10000</td><td>Zoya</td></tr> </table>	1	Aman	2	Neha	3	Rahul	4	Priya	...	...	10000	Zoya	<table border="1"> <tr><td>Aman</td><td>→</td><td>Row 1</td></tr> <tr><td>Neha</td><td>→</td><td>Row 2</td></tr> <tr><td>Priya</td><td>→</td><td>Row 4</td></tr> <tr><td>Rahul</td><td>→</td><td>Row 3</td></tr> <tr><td>...</td><td>→</td><td>...</td></tr> </table>	Aman	→	Row 1	Neha	→	Row 2	Priya	→	Row 4	Rahul	→	Row 3	...	→	...
1	Aman																											
2	Neha																											
3	Rahul																											
4	Priya																											
...	...																											
10000	Zoya																											
Aman	→	Row 1																										
Neha	→	Row 2																										
Priya	→	Row 4																										
Rahul	→	Row 3																										
...	→	...																										

Directly goes to matching row

6. Important Notes

- Indexes improve read performance but may slow down INSERT, UPDATE, DELETE operations.
- Do not create indexes on small tables.
- Too many indexes can use more storage.
- Choose columns with high cardinality (unique values).

Key Points

- Indexes help in faster data retrieval.
- Use indexes on columns used in WHERE, JOIN, ORDER BY, GROUP BY.
- Balance is key - more indexes are not always better.

Q7. What are Stored Procedures?

Answer:

A stored procedure is a precompiled SQL code stored in the database. It can accept input parameters, perform operations and return results.

1. Why use Stored Procedures?

- Reusability - Write once, use many times.
- Performance - Precompiled execution plan.
- Security - Grant execute permission, not direct table access.
- Maintainability - Centralized logic.
- Reduced network traffic.

2. Syntax

```
CREATE PROCEDURE procedure_name
(
    parameter1 datatype,
    parameter2 datatype,
    ...
)
BEGIN
    -- SQL statements
END;
```

3. Example - Create Stored Procedure

```
CREATE PROCEDURE GetEmployeeByDept
(
    IN dept_id INT
)
BEGIN
    SELECT EmpID, EmpName, Salary
    FROM Employee
    WHERE DeptID = dept_id;
END;
```

4. Call / Execute Stored Procedure

- MySQL :
CALL GetEmployeeByDept(10);
- SQL Server :
EXEC GetEmployeeByDept @dept_id = 10;
- PostgreSQL :
CALL GetEmployeeByDept(10);
- Oracle :
BEGIN GetEmployeeByDept(10); END;

5. Stored Procedure with OUT Parameter

```
CREATE PROCEDURE GetEmployeeCount
(
    OUT emp_count INT
)
BEGIN
    SELECT COUNT(*) INTO emp_count
    FROM Employee;
END;
```

6. Call Procedure with OUT Parameter

- MySQL :
CALL GetEmployeeCount(@cnt);
SELECT @cnt;
- SQL Server :
DECLARE @cnt INT;
EXEC GetEmployeeCount @emp_count = @cnt OUTPUT;
SELECT @cnt;

7. Important Notes

- Use meaningful names for procedures.
- Handle exceptions and validations inside procedure.
- Use parameters to make procedures dynamic.
- Avoid complex logic in procedures; keep it clean.

Pro Tip:

Stored procedures are widely used in real-world applications for business logic, reporting and data operations.

Quick Summary

Feature	Benefit	Example Use
Reusability	Write once, use many times	Get employee list, generate report
Performance	Precompiled, faster execution	Frequent queries, large data
Security	Control access, hide logic	Allow EXECUTE, not table access

Q8. What are Triggers?

Answer:

A trigger is a special type of stored procedure that automatically executes in response to certain events on a table.

It is used to enforce rules, maintain audit logs and ensure data integrity.

1. Key Points

- Triggers are automatically executed.
- They are associated with a table.
- They fire on an event (INSERT, UPDATE, DELETE).
- They can be BEFORE or AFTER the event.
- They help in auditing and validation.

2. Trigger Types

Type	Description
BEFORE Trigger	Fires before the event occurs.
AFTER Trigger	Fires after the event occurs.
INSTEAD OF Trigger	Fires instead of the event.
ROW Level Trigger	Fires for each row affected.
STATEMENT Level Trigger	Fires once per statement.

3. Syntax

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF} {INSERT | UPDATE | DELETE}
ON table_name
{FOR EACH ROW | FOR EACH STATEMENT}
BEGIN
    -- SQL statements
END;
```

4. Example - Create Trigger

```
CREATE TRIGGER trg_emp_audit
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Audit
        (EmpID, EmpName, Action, ActionDate)
    VALUES (NEW.EmpID, NEW.EmpName, 'INSERT', NOW());
END;
```

5. Example - How it Works

Insert into Employee
(EmpID, EmpName, Salary)
Values (1, 'Aman', 60000);



Trigger (trg_emp_audit) Fires
Automatically inserts into
Employee_Audit table.

6. Common Use Cases

- Maintain audit trail of data changes.
- Enforce business rules and validations.
- Update summary or derived tables.
- Prevent unwanted data modifications.

7. Important Notes

- Triggers can affect performance if overused.
- Avoid using triggers for complex business logic.
- Be careful with recursive triggers (infinite loop).
- Test triggers thoroughly before using in production.

8. Quick Summary

Aspect	Details
Definition	A trigger is a special procedure that runs automatically on table events.
Events	INSERT, UPDATE, DELETE
Timing	BEFORE, AFTER, INSTEAD OF
Level	Row Level or Statement Level
Purpose	Enforce rules, auditing, maintain integrity

Pro Tip:

Use triggers carefully.
Overuse can make your
database hard to
maintain and debug.

Q9. What is Normalization?

Answer:

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

It divides large tables into smaller, related tables and defines relationships between them.

1. Why Normalize?

- Reduce data redundancy.
- Avoid update anomalies (insert, update, delete).
- Improve data consistency and integrity.
- Save storage space.
- Make database easier to maintain.

2. Normal Forms (1NF to 3NF)

Normal Form	Description
1NF	All attributes are atomic (no repeating groups). Each row is unique.
2NF	In 1NF and all non-key attributes are fully functionally dependent on the primary key.
3NF	In 2NF and no transitive dependency. Non-key attributes depend only on the key.

3. Example - Employee Table (Unnormalized)

Table: Employee_Details

EmpID	EmpName	DeptID	DeptName	DeptLocation	ManagerID	ManagerName
1	Aman	10	IT	Gurugram	100	Raj
2	Neha	20	HR	Delhi	101	Sneha
3	Rahul	10	IT	Gurugram	100	Raj

Problems:

- DeptName and DeptLocation are repeated.
- If DeptLocation changes, multiple rows must be updated.
- Insert or delete anomalies may occur.

4. After Normalization

1NF: Atomic Values

Table: Employee

EmpID	EmpName	DeptID	ManagerID
1	Aman	10	100
2	Neha	20	101
3	Rahul	10	100

2NF: Remove Partial Dependency

Table: Department

DeptID	DeptName	DeptLocation
10	IT	Gurugram
20	HR	Delhi

3NF: Remove Transitive Dependency

Table: Manager

ManagerID	ManagerName
100	Raj
101	Sneha

Relationships:

- Employee.DeptID → Department.DeptID
- Employee.ManagerID → Manager.ManagerID

6. Important Notes

- Do not over-normalize; it may impact performance.
- Denormalization can be used in reporting or high-read systems.
- Always balance between performance and data integrity.
- Use normalization for OLTP systems, denormalization for OLAP/reporting.

5. Normalization Levels

- 1NF → Atomic values, no repeating groups
- 2NF → Remove partial dependencies
- 3NF → Remove transitive dependencies
- BCNF → Stronger version of 3NF
- 4NF → Handle multi-valued dependencies
- 5NF → Handle join dependencies

7. Quick Summary

NF	Focus	Key Rule	Example Benefit
1NF	Atomicity	No repeating groups	Eliminate duplicate or nested data
2NF	Remove partial dependency	Non-key depends on full key	Avoid partial redundancy
3NF	Remove transitive dependency	Non-key depends only on key	Improve consistency and integrity

Q10. What is ACID Property in SQL?

Answer:

ACID is a set of properties that guarantee reliable processing of database transactions. It helps maintain data consistency even in case of failures.

1. The Four ACID Properties

Property	Description
Atomicity	All operations in a transaction succeed or none at all.
Consistency	Transaction brings database from one valid state to another valid state.
Isolation	Concurrent transactions do not interfere with each other.
Durability	Once committed, data remains saved even if system fails.

2. Example - Bank Transfer

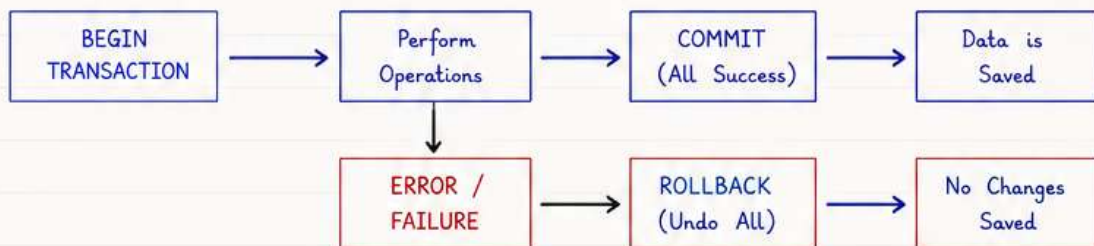
Transfer ₹1000 from Account A to Account B should follow ACID.

Steps:

1. Deduct ₹1000 from Account A
2. Add ₹1000 to Account B
3. Commit

If any step fails → entire transaction is rolled back.

3. How ACID Works in a Transaction



4. SQL Commands for Transactions

- **BEGIN TRANSACTION;** -- Start transaction
- **COMMIT;** -- Save changes permanently
- **ROLLBACK;** -- Undo changes
- **SAVEPOINT name;** -- Set save point
- **ROLLBACK TO name;** -- Rollback to save point
- **RELEASE SAVEPOINT name;** -- Remove save point

5. Example - Using Transactions

```

BEGIN TRANSACTION;

UPDATE Account SET Balance = Balance - 1000
WHERE AccNo = 'A';

UPDATE Account SET Balance = Balance + 1000
WHERE AccNo = 'B';

COMMIT;
  
```

6. Isolation Levels

Level	Description
READ UNCOMMITTED	Can read uncommitted changes (dirty read possible).
READ COMMITTED	Can read only committed data.
REPEATABLE READ	Same data is read every time in a transaction.
SERIALIZABLE	Highest level; transactions run one after another.

7. Important Notes

- ACID ensures data reliability.
- ROLLBACK is important on failure.
- Isolation level affects performance.
- Use transactions for critical operations.

8. Quick Summary

Property	Focus	Key Benefit	Example
Atomicity	All or nothing	No partial updates	Bank transfer
Consistency	Valid state	Data integrity	Check constraints
Isolation	No interference	Accurate concurrent access	Multiple users
Durability	Permanent save	Survive system failure	Committed data

COMMENT SQL FOR

SQL NOTES

FOLLOW 
FOR PART 3 
ADVANCE TOPIC



TARGET 3K LIKES 

I WILL SHARE PART 3 



SAVE



SHARE



FOLLOW



Aman Views

